PreTest AI — Available Code Backup + Submission Guide

This document preserves the code/integration material that was actually available in the project backup. It deliberately does not invent the unseen byte-for-byte contents of local files.

```
PreTest AI — Master Code Backup
```

Latest Agent + Incident Engine integration | 29 September 2026

Project structure

```
PreTestAI/
■
■■■ .env
■■■ .env.example
■■■ requirements.txt
■
■■■ backend/
■
■■■ main.py
■
■■■ main_backup.py
■
■■■ models.py
■
■■■ llm_client.py
■
■■■ domain_engine.py
■
■■■ question_engine.py
■
■■■ testplan_engine.py
■
■■■ calculation_engine.py
■
■■■ risk_engine.py
■
■■■ report_generator.py
■
■■■ hindsight_engine.py
■
■■■ agent_engine.py
■
■■■ incident_engine.py
■
■■■ demo_data.py
■
■■■ frontend/
■■■ index.html
```

Latest Agent Engine — backend/agent_engine.py

```
"""
PreTest AI - Agent Engine
Conversation and project-continuity layer.
This engine does not replace Hindsight Memory.
It interprets a new user message against an existing project,
detects meaningful changes, identifies affected areas, and
prepares a reassessment request.
"""
from typing import Dict, Any
from .calculation_engine import DeterministicCalculationEngine
class PreTestAgent:
"""
Agent layer for continuing an existing PreTest AI project.
Hindsight remains responsible for version memory/history.
"""
CONTINUATION_PHRASES = [
"continue", "previous", "earlier", "before",
"last assessment", "same project", "same idea",
"my project", "our project", "as we discussed",
"as discussed", "change", "changed", "instead",
"update", "updated", "modify", "modified",
"new feature", "added", "remove", "removed",
"replace", "replaced", "now", "actually",
]
UNKNOWN_PHRASES = [
"not decided", "haven't decided", "have not decided",
"unknown", "tbd", "unclear", "not sure",
"don't know", "do not know",
]
def __init__(self):
self.name = "PreTest AI Agent"
self.version = "1.0"
def get_status(self):
return {
```

```
"agent": self.name,
"version": self.version,
"mode": "Deterministic Project Continuity Agent",
"memory_provider": "Hindsight-compatible",
"supports": [
"project continuation",
"change detection",
"impact mapping",
"selective reassessment",
],
}
def is_continuation_message(self, message, existing_project=None):
if not message:
return False
text = message.lower().strip()
if existing_project:
return True
return any(phrase in text for phrase in self.CONTINUATION_PHRASES)
def detect_parameter_changes(self, old_parameters, new_message):
new_parameters = DeterministicCalculationEngine.extract_numbers_and_units(new_message)
changed = {}
parameter_labels = {
"unit_cost": "Unit Cost",
"selling_price": "Selling Price",
"production_volume": "Production Volume",
}
for key, label in parameter_labels.items():
if key not in new_parameters:
continue
new_value = new_parameters[key]
old_value = old_parameters.get(key)
if old_value is None:
changed[key] = {
"label": label,
"old": None,
"new": new_value,
"change_type": "NEW_PARAMETER",
}
elif old_value != new_value:
changed[key] = {
"label": label,
"old": old_value,
"new": new_value,
"change_type": "MODIFIED",
}
return changed
def detect_textual_changes(self, old_text, new_message):
text = new_message.lower()
changes = []
keyword_groups = {
"target_users": [
"target users", "customers", "customer", "users",
"patients", "students", "farmers", "hospitals", "clinics"
],
"technology": [
"technology", "platform", "software", "hardware",
"sensor", "battery", "model", "ai model", "algorithm",
"cloud", "on-premise", "on premise"
],
"scale": [
"scale", "capacity", "production", "volume",
"units per day", "units/day", "users per day"
],
"location": [
"location", "city", "state", "country", "region", "move to",
"launch in"
],
"timeline": [
"deadline", "timeline", "within", "days", "weeks",
"months", "launch date"
],
"regulatory": [
"approval", "license", "licence", "regulation",
"regulatory", "compliance", "certification"
],
"safety": [
"safety", "hazard", "failure", "incident", "accident",
"temperature", "overheating", "thermal"
],
}
for category, keywords in keyword_groups.items():
matched = [keyword for keyword in keywords if keyword in text]
if matched:
changes.append({
"category": category,
"matched_terms": ", ".join(matched),
"source": "New user message",
})
return changes
def map_affected_areas(self, parameter_changes, textual_changes, new_message):
affected = set()
if "unit_cost" in parameter_changes:
affected.update([
```

```
"Financial Analysis", "Unit Economics",
"Gross Margin", "Financial Risk"
])
if "selling_price" in parameter_changes:
affected.update([
"Financial Analysis", "Pricing", "Unit Economics",
"Gross Margin", "Market Assumptions"
])
if "production_volume" in parameter_changes:
affected.update([
"Scale", "Operational Feasibility", "Capacity",
"Unit Economics", "Scalability Risk"
])
for item in textual_changes:
category = item["category"]
if category == "target_users":
affected.update([
"Target Users", "Market Assumptions",
"Operational Requirements", "Usability"
])
elif category == "technology":
affected.update([
"Technical Feasibility", "Architecture",
"Reliability", "Security"
])
elif category == "scale":
affected.update([
"Scalability", "Operations",
"Financial Analysis", "Capacity"
])
elif category == "location":
affected.update([
"Operational Environment", "Regulatory Requirements",
"Logistics", "External Factors"
])
elif category == "timeline":
affected.update([
"Timeline Feasibility", "Operations",
"Resource Requirements"
])
elif category == "regulatory":
affected.update([
"Compliance", "Regulatory Risk",
"Required Validation"
])
elif category == "safety":
affected.update([
"Safety", "Risk Register",
"Failure Scenarios", "Validation Tests"
])
incident_words = [
"incident", "failure", "failed", "unexpected",
"problem", "issue", "overheating", "crash",
"leak", "breakdown"
]
if any(word in new_message.lower() for word in incident_words):
affected.update([
"Incident Analysis", "Failure Scenarios",
"Risk Register", "Required Validation"
])
return sorted(affected)
def detect_unknowns(self, message):
text = message.lower()
return [phrase for phrase in self.UNKNOWN_PHRASES if phrase in text]
def process_message(self, project_id, message, project_data=None):
message = (message or "").strip()
if not message:
return {
"status": "INVALID_MESSAGE",
"agent_message": "Please provide a message about the project.",
"project_id": project_id,
}

project_data = project_data or {}
profile = project_data.get("profile")
if hasattr(profile, "model_dump"):
profile_data = profile.model_dump()
elif isinstance(profile, dict):
profile_data = profile
else:
profile_data = {}
old_parameters = project_data.get("numerical_parameters", {})
old_text = project_data.get("raw_text", "")
parameter_changes = self.detect_parameter_changes(
old_parameters, message
)
textual_changes = self.detect_textual_changes(
old_text, message
)
affected_areas = self.map_affected_areas(
parameter_changes, textual_changes, message
)
unknowns = self.detect_unknowns(message)
continuation = self.is_continuation_message(
```

```
message, project_data
)
has_changes = bool(parameter_changes or textual_changes)
if unknowns:
status = "VALIDATION_REQUIRED"
elif has_changes:
status = "REASSESSMENT_REQUIRED"
elif continuation:
status = "PROJECT_CONTINUATION"
else:
status = "NEW_INFORMATION"
if status == "REASSESSMENT_REQUIRED":
changed_names = [
item["label"] for item in parameter_changes.values()
]
changed_names.extend(
item["category"] for item in textual_changes
)
agent_message = (
"I remember this project. I detected changes in "
+ ", ".join(changed_names)
+ ". The existing assessment should not be discarded. "
"I will reassess the affected calculations, risks, "
"assumptions, and validation tests."
)
elif status == "VALIDATION_REQUIRED":
agent_message = (
"I remember this project, but the new information "
"contains an unresolved assumption. I have marked it "
"for validation before treating the affected conclusion "
"as reliable."
)
elif status == "PROJECT_CONTINUATION":
project_name = profile_data.get(
"project_name", "this project"
)
agent_message = (
f"Yes, I remember {project_name}. I am continuing "
"from the existing project assessment and its stored "
"version history."
)
else:
agent_message = (
"I have recorded the new information against the "
"current project context. No major parameter change "
"was detected yet."
)
return {
"status": status,
"project_id": project_id,
"agent": self.name,
"agent_message": agent_message,
"is_continuation": continuation,
"changes_detected": has_changes,
"parameter_changes": parameter_changes,
"textual_changes": textual_changes,
"affected_areas": affected_areas,
"unknown_information": unknowns,
"requires_reassessment": status == "REASSESSMENT_REQUIRED",
"requires_validation": status == "VALIDATION_REQUIRED",
"previous_project_available": bool(project_data),
"previous_version": profile_data.get("version", 1),
}

agent_engine = PreTestAgent()
```

Latest Incident Engine — backend/incident_engine.py

```
"""
PreTest AI - Incident Engine
Detects new incidents or unexpected observations in an existing project.
Works together with Agent Engine and Hindsight Memory.
"""
from typing import Dict, Any, List
from datetime import datetime
import uuid

class IncidentEngine:
INCIDENT_TERMS = [
"incident", "failure", "failed", "crash", "problem",
"issue", "unexpected", "malfunction", "overheating",
"leak", "breakdown", "error", "accident",
"not working", "stopped working", "performance dropped"
]
CATEGORY_TERMS = {
"Safety": [
"safety", "accident", "overheating", "hazard",
"fire", "leak"
],
"Technical": [
"error", "crash", "failure", "malfunction",
"sensor", "software", "hardware"
],
```

```
"Performance": [
"slow", "performance dropped", "accuracy dropped",
"delay", "latency", "unstable"
],
"Operational": [
"production", "delay", "breakdown", "capacity",
"process", "maintenance"
],
"Financial": [
"cost", "price", "loss", "budget",
"expense", "revenue"
],
"Regulatory": [
"license", "licence", "approval", "compliance",
"regulation", "certification"
],
"Security": [
"security", "attack", "unauthorized",
"data breach", "privacy", "access"
]
}
def __init__(self):
self.name = "PreTest AI Incident Engine"
self.version = "1.0"
def get_status(self):
return {
"engine": self.name,
"version": self.version,
"mode": "Deterministic Incident Analysis"
}
def detect_incident(self, message: str):
text = (message or "").lower()
matched_terms = [
term for term in self.INCIDENT_TERMS
if term in text
]
return {
"detected": bool(matched_terms),
"matched_terms": matched_terms
}
def classify_incident(self, message: str) -> List[str]:
text = (message or "").lower()
categories = []
for category, keywords in self.CATEGORY_TERMS.items():
if any(keyword in text for keyword in keywords):
categories.append(category)
return categories
def map_affected_areas(self, categories: List[str]):
mapping = {

"Safety": [
"Safety", "Risk Register",
"Failure Scenarios", "Validation Tests"
],
"Technical": [
"Technical Feasibility", "Reliability",
"Architecture", "Validation Tests"
],
"Performance": [
"Performance", "Reliability",
"Performance Tests"
],
"Operational": [
"Operations", "Operational Feasibility",
"Capacity"
],
"Financial": [
"Financial Analysis", "Unit Economics",
"Financial Risk"
],
"Regulatory": [
"Compliance", "Regulatory Requirements",
"Regulatory Risk"
],
"Security": [
"Security", "Privacy",
"Security Validation"
]
}
affected = set()
for category in categories:
affected.update(mapping.get(category, []))
return sorted(affected)
def build_incident(
self,
project_id: str,
message: str,
project_data: Dict[str, Any] = None
):
project_data = project_data or {}
detection = self.detect_incident(message)
if not detection["detected"]:
return {
"status": "NO_INCIDENT_DETECTED",
```

```
"project_id": project_id,
"incident_detected": False,
"requires_reassessment": False
}
categories = self.classify_incident(message)
affected_areas = self.map_affected_areas(categories)
profile = project_data.get("profile")
if hasattr(profile, "model_dump"):
profile_data = profile.model_dump()
elif isinstance(profile, dict):
profile_data = profile
else:
profile_data = {}
previous_version = profile_data.get("version", 1)
incident_id = "INC-" + uuid.uuid4().hex[:8].upper()
tests_to_repeat = []
for category in categories:
if category == "Safety":
tests_to_repeat.append(
"Safety and failure validation"
)
elif category == "Technical":
tests_to_repeat.append(
"Technical feasibility validation"
)
elif category == "Performance":
tests_to_repeat.append(
"Performance and reliability testing"
)
elif category == "Operational":
tests_to_repeat.append(
"Operational feasibility testing"
)
elif category == "Financial":
tests_to_repeat.append(
"Financial and unit-economics checks"
)
elif category == "Regulatory":
tests_to_repeat.append(
"Regulatory and compliance review"
)
elif category == "Security":
tests_to_repeat.append(
"Security and privacy validation"
)
if not tests_to_repeat:
tests_to_repeat.append("General project validation")
return {
"status": "INCIDENT_DETECTED",
"project_id": project_id,
"incident_id": incident_id,
"incident_detected": True,
"detected_terms": detection["matched_terms"],
"categories": categories,
"affected_areas": affected_areas,
"tests_to_repeat": tests_to_repeat,
"previous_version": previous_version,
"requires_reassessment": True,
"preserve_previous_findings": True,
"detected_at": datetime.utcnow().isoformat(),
"message": (
"A new incident or unexpected observation was detected. "
"The previous assessment is preserved while affected "
"assumptions, risks, calculations and tests are reassessed."
)
}
incident_engine = IncidentEngine()
```

main.py — required imports

```
# Add these two imports beside the existing Hindsight import:
from .hindsight_engine import hindsight_manager
from .agent_engine import agent_engine
from .incident_engine import incident_engine
```

main.py — Agent endpoint

```
@app.post("/api/agent/message")
def agent_message(request: HindsightChatRequest):
session = PROJECT_SESSIONS.get(request.project_id)
if not session:
raise HTTPException(
status_code=404,
detail="Project session not found."
)
agent_result = agent_engine.process_message(
project_id=request.project_id,
message=request.user_message,
project_data=session
)
```

```
incident_result = incident_engine.build_incident(
project_id=request.project_id,
message=request.user_message,
project_data=session
)
if incident_result.get("incident_detected"):
session.setdefault("incident_history", [])
session["incident_history"].append(incident_result)
session.setdefault("agent_history", [])
session["agent_history"].append(agent_result)
return {
"status": "AGENT_ANALYSIS_COMPLETE",
"agent": agent_result,
"incident": incident_result,
"project_id": request.project_id,
"previous_version": agent_result.get("previous_version"),
"requires_reassessment": (
agent_result.get("requires_reassessment", False)
or incident_result.get("requires_reassessment", False)
)
}
```

main.py — current integration core

```
# Current main.py core structure already in the project
import os
from typing import Dict, Any
from fastapi import FastAPI, HTTPException, Body
from fastapi.responses import FileResponse
from fastapi.middleware.cors import CORSMiddleware
from .models import (
IdeaRequest, ProjectProfile, QuestionItem, AnswerSubmission,
PreImplementationReport, VersionDiff, HindsightChatRequest
)
from .llm_client import llm_client
from .domain_engine import extract_domain_and_profile
from .question_engine import (
generate_adaptive_questions,
generate_progressive_followup
)
from .testplan_engine import generate_dynamic_test_plan
from .calculation_engine import DeterministicCalculationEngine
from .risk_engine import DeterministicRiskAndAssumptionsEngine
from .hindsight_engine import hindsight_manager
from .agent_engine import agent_engine
from .incident_engine import incident_engine
from .report_generator import compile_pre_implementation_report
# Existing PROJECT_SESSIONS, /api/assess, /api/questions/answer,
# /api/profile/update, /api/analyze, Hindsight routes and demo routes
# remain in place.
```

PowerShell / Python verification commands

```
# Activate environment (optional if using the full path)
.\.venv\Scripts\Activate.ps1
# Check the two new engines
.\.venv\Scripts\python.exe -m py_compile .\backend\agent_engine.py
.\.venv\Scripts\python.exe -m py_compile .\backend\incident_engine.py
# Check the complete backend
.\.venv\Scripts\python.exe -m compileall .\backend
# Start PreTest AI
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
# Health check from a second PowerShell window
Invoke-WebRequest http://127.0.0.1:8000/api/health
```

Backup note

IMPORTANT BACKUP NOTE

```
This PDF contains the latest Agent Engine and Incident Engine code and the exact integration changes made during this
session.
The older project files (models.py, domain_engine.py, question_engine.py, testplan_engine.py, calculation_engine.py,
risk_engine.py, hindsight_engine.py, report_generator.py, demo_data.py, llm_client.py and the complete original frontend)
were created/edited in the user's local project. Their complete byte-for-byte contents are not available as attachments in
this
chat, so this PDF does NOT pretend to reproduce unseen lines.
When you return from college, upload the complete PreTestAI folder (ZIP) here if you want a second PDF containing every
existing file line-by-line, exactly as it currently exists.
The latest changes are:
1. agent_engine.py added.
2. incident_engine.py added.
3. main.py imports both engines.
4. /api/agent/message endpoint added.
5. main_backup.py created before modifying main.py.
6. Whole backend compile check passed without SyntaxError.
```

GitHub upload checklist

• Upload the actual runnable project files from C:\Users\DELL\PreTestAI, not only this PDF.

• Include backend/, frontend/, requirements.txt, .env.example, and a README.md.

• Do NOT upload .env when it contains an API key or other secret. Use .env.example with placeholders.

• This PDF can be added as documentation, for example under docs/.

Reddit next step

Before posting, check the target subreddit rules. A simple project-showcase post can use: Problem → What PreTest AI does → Agent + Hindsight memory behavior → live demo → GitHub link → request for feedback.

Suggested title: I built PreTest AI to test a real-world idea before implementation — with agent memory and selective reassessment